

AdiVaani Hiring Assignment

Prepared by Sneha Oram

June 26, 2026

Part-2: Language Model Pretraining

Method 1: BERT-like Encoder Model

Data pre-processing

- The datasets used to pretrain the model are data split for Hindi and Marathi sentences from huggingface datasets: [ai4bharat/IndicCorpV2](#)
- For pretraining, 10% of the Hindi sentences and 10% of the Marathi sentences are chosen.

Training Design

- Encoder model is trained from scratch using the Masked Language Modeling (MLM) objective in PyTorch.
- Before the start of the training, the data is set in a line and divided into continuous chunks of sequence length 128.
- Apart from the given implementation details, including Grouped Query Attention (GQA), Rotary Positional Embeddings (RoPE), and RMSNorm, several modern architectural and optimization components are also included, such as:
 - Dynamic Masking
 - Gradient Accumulation
 - Cosine Learning Rate Scheduler with Warmup
 - Weight Tying
 - Distributed Training with NCCL backend
- The key features include:
 - MLM pretraining objective with runtime dynamic masking
 - BF16 mixed precision training using torch.autocast
 - Multi-GPU training with PyTorch DDP
 - Gradient clipping and accumulation
 - Cosine decay scheduler with warmup
 - Periodic checkpoint saving

Component	Value
Hidden Size	512
Encoder Layers	16
Attention Heads	8
KV Heads (GQA)	2
Sequence Length	128
Vocabulary Size	32000
MLM Mask Ratio	40%

Table 1: Model Configuration for BERT encoder model.

Setting	Value
GPUs	$2 \times 80\text{GB}$
Micro Batch Size / GPU	8
Gradient Accumulation	16
Effective Global Batch Size	256
Optimizer	AdamW
Learning Rate	$3.00\text{E-}04$
Scheduler	Cosine Decay + Warmup
Precision	BF16

Table 2: Training setup BERT encoder model.

Results and Analysis



Figure 1: Training loss over the micro-steps in BERT like encoder model.

The training loss for the Encoder model is shown in Figure 1. The training is conducted with 1 epoch, with 575 micro-steps.

- The plot of the training loss suggests that although the training loss fluctuates, it slowly decreases over the micro-steps.

Method 2: GPT-2 style Decoder Model

The same set-up for the training set is chosen as Method §.

- The datasets used to pretrain the model are data split for Hindi and Marathi sentences from huggingface datasets: [ai4bharat/IndicCorpV2](#)
- For pretraining, 10% of the Hindi sentences and 10% of the Marathi sentences are chosen.

Training Design

- Decoder model is trained from scratch using the next-token prediction objective in PyTorch.

Component	Value
Hidden Size	512
Decoder Layers	16
Attention Heads	8
KV Heads (GQA)	4
Sequence Length	128
Vocabulary Size	32000
Dropout	0.1

Table 3: Model Configuration for GPT2 decoder model.

Setting	Value
GPUs	$2 \times 80\text{GB}$
Micro Batch Size / GPU	8
Gradient Accumulation	16
Effective Global Batch Size	256
Optimizer	AdamW
Learning Rate	$3.00\text{E-}04$
Scheduler	Cosine Decay + Warmup
Precision	BF16

Table 4: Training setup for GPT2 decoder model.

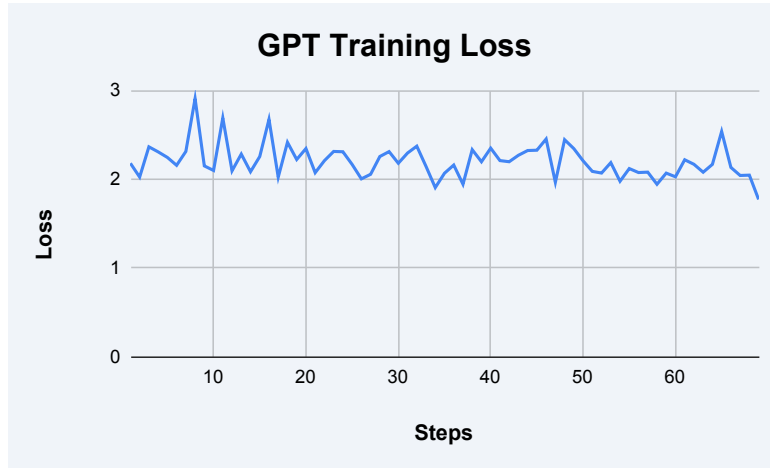


Figure 2: Training loss over the micro-steps in GPT2 like decoder model.

- Before the start of the training, the data is set in a line and divided into continuous chunks of sequence length 128.
- Apart from the given implementation details, including Grouped Query Attention (GQA), Rotary Positional Embeddings (RoPE), and RMSNorm, several modern architectural and optimization components are also included, such as:
 - SwiGLU Feed Forward Networks
 - Gradient Accumulation
 - Cosine Learning Rate Scheduler with Warmup
 - Weight Tying
 - Distributed Multi-GPU Training
- The key features include:
 - Autoregressive next-token prediction training
 - BF16 mixed precision training
 - Multi-GPU training using PyTorch DDP
 - Gradient clipping and accumulation
 - Cosine decay scheduler with warmup
 - Periodic checkpoint saving

Results and Analysis

The training loss for the Decoder model is shown in Figure 2. The training is conducted with 1 epoch, with 581 micro-steps.

- The training loss plot suggests that, although the loss exhibits some fluctuations, it demonstrates a gradual decreasing trend over the micro-steps.

Future Works

- The evaluation of the individual pretrained models, along with the joint training for the translation task, is considered future work.

Here is the GitHub link for reproducibility: [Pretraining](#)